FIAP

GRADUAÇÃO

45697056

ES
Desenvolvimento Mobile

Prof. Adeilton Meneses

profadeilton.meneses@fiap.com.br

Persistência de Dados com Async Storage no React Native

45697056

- **Conceitos:** Por que e quando persistir dados localmente.
- **Instalação:** Configurando a biblioteca com o Expo.
- **Operações CRUD:** `setItem`, `getItem`, `removeItem`.
- **Estruturas de Dados:** Salvando e lendo Objetos/Arrays (JSON).
- **Integração:** Uso prático com `useState` e `useEffect`
- **Boas Práticas & Segurança.**

Documentação: https://docs.expo.dev/versions/latest/sdk/async-storage/

Persistência de Dados com Async Storage no React Native

45697056

Por que precisamos Armazenar Dados no dispositivo?

- "Em um aplicativo web, temos o *LocalStorage* do navegador. No mobile, precisamos de algo semelhante, mas que seja **persistente** mesmo quando o app é fechado completamente ou o telefone é reiniciado."
- "A persistência local garante a **continuidade da experiência do usuário** (UX) e reduz a dependência de chamadas constantes ao servidor."

Documentação: https://docs.expo.dev/versions/latest/sdk/async-storage/

Persistência de Dados com Async Storage no React Native

45697056

Casos de Uso Essenciais:

- **Configurações de UI:** Manter o *Modo Escuro/Claro* ativo.
- **Estado de Sessão:** Lembrar que o usuário já fez o *onboarding* ou se ele está logado (usando tokens/IDs).
- **Dados Offline:** Guardar rascunhos de mensagens ou listas de tarefas para uso sem internet.

Persistência de Dados com Async Storage no React Native

45697056

Contraste Rápido (Quando escolher o quê):

- `AsyncStorage` **(Nosso Foco):** Excelente para **pequenos volumes** de dados não sensíveis (preferências, IDs, pequenas configurações). Simplicidade máxima.

- `expo-secure-store:` **Obrigatoriamente** para dados sensíveis (ex: Tokens de Autenticação, Credenciais). Ele usa o *Keychain* do iOS e *Keystore* do Android para criptografia nativa.

- **SQLite/Expo-SQLite:** Necessário apenas quando você precisa de **modelos de dados complexos**, consultas relacionais ou grandes volumes de informação estruturada.

Introdução a Async Storage no React Native

45697056

O que é o `AsyncStorage`?

- **Descrição:** A solução padrão de armazenamento local para React Native.
  - **Assíncrono:** Operações não travam a *thread* principal (UI fluida).
  - **Persistente:** Os dados sobrevivem ao fechar e reabrir o app.
  - **Chave-Valor:** Armazena pares simples de `(string, string)`.

- **Dependência:** No Expo, usamos o pacote comunitário: `@react-native-async-storage/async-storage`.

Instalando a biblioteca

45697056

- react-native-async-storage/async-storage: é uma biblioteca da comunidade para React Native que fornece um **sistema de armazenamento persistente, assíncrono e de chave-valor**.

```
npx expo install @react-native-async-storage/async-storage
```

```
import AsyncStorage from '@react-native-async-storage/async-storage';
```

Operações Fundamentais (CRUD Básico)

45697056

Salvando Dados (`setItem`)

- **Descrição:** Inserir ou atualizar um valor associado a uma chave. É uma **Promise**, logo, usamos `async/await`.
- **Regra de Ouro:** O `AsyncStorage` só armazena **strings**.
- **Exemplo de Código (Salvar uma String):**

```
const CHAVE_NOME = '@minhaApp:nomeUsuario';

const salvarNome = async (nome) => {
  try {
    await AsyncStorage.setItem(CHAVE_NOME, nome);
    console.log('Nome salvo com sucesso!');
  } catch (error) {
    console.error('Erro ao salvar:', error);
  }
};
```

Operações Fundamentais (CRUD Básico)

45697056

Lendo Dados (`getItem`)

- **Descrição:** Recuperar o valor de uma chave específica. Retorna `null` se a chave não existir.
- **Exemplo de Código (Ler uma String):**

```
const lerNome = async () => {
  try {
    const nomeSalvo = await AsyncStorage.getItem(CHAVE_NOME);
    if (nomeSalvo !== null) {
      console.log('Nome lido:', nomeSalvo);
      // Aqui você atualizaria o estado do seu componente
      // setNome(nomeSalvo);
    } else {
      console.log('Nenhum nome encontrado.');
    }
  } catch (error) {
    console.error('Erro ao ler:', error);
  }
};
```

Operações Fundamentais (CRUD Básico)

45697056

Removendo e Limpando Dados

- **Descrição:**
  - `removeItem(key)`: Remove uma chave específica.
  - `clear()`: Remove **TODOS** os dados armazenados pelo seu aplicativo. Cuidado ao usar!
- **Exemplo de Código (Remoção e Logout):**

```
const limparDados = async () => {
  try {
    // 1. Remover uma chave específica:
    // await AsyncStorage.removeItem(CHAVE_NOME);

    // 2. Limpar TUDO (Exemplo para Logout):
    await AsyncStorage.clear();

    console.log('Dados removidos/limpos!');
    // Navegar para a tela de Login
  } catch (error) {
    console.error('Erro ao limpar:', error);
  }
};
```

Operações Avançadas e Gerenciamento de Estado

45697056

Armazenamento de Objetos e Arrays (A Serialização)

- **Descrição:** Como salvar dados complexos (objetos/arrays) se o `AsyncStorage` só aceita *strings*.
  - **Salvar:** Use `JSON.stringify(objeto)` antes de chamar `setItem`.
  - **Ler:** Use `JSON.parse(string)` após chamar `getItem`.
- **Exemplo de Código (Objeto):**

```
const configuracao = { tema: 'dark', notificacoes: true };
const CHAVE_CONFIG = '@minhaApp:config';

// SALVAR
const salvarConfig = async () => {
  const jsonValue = JSON.stringify(configuracao);
  await AsyncStorage.setItem(CHAVE_CONFIG, jsonValue);
};

// LER
const lerConfig = async () => {
  const jsonValue = await AsyncStorage.getItem(CHAVE_CONFIG);
  const configLida = jsonValue != null ? JSON.parse(jsonValue) : null;
  console.log(configLida.tema); // 'dark'
};
```

Operações Avançadas e Gerenciamento de Estado

45697056

Atualizando Dados Parcialmente (`mergeItem`)

- **Descrição:** Útil para atualizar apenas *algumas* propriedades de um objeto já existente, sem precisar ler, modificar e reescrever o objeto inteiro.
  - **Sintaxe:** `mergeItem(key, valueString)`
  - O novo objeto (em formato string) é mesclado com o objeto existente sob a mesma chave.
- **Exemplo de Código (Atualizando Preferência):**

```
const atualizarNotificacoes = async (novoValor) => {
  const novaConfigParcial = JSON.stringify({ notificacoes: novoValor });
  try {
    // Se já existir um objeto em @minhaApp:config, ele mescla.
    await AsyncStorage.mergeItem(CHAVE_CONFIG, novaConfigParcial);
    console.log('Notificação atualizada.');
  } catch (error) {
    console.error('Erro ao mesclar:', error);
  }
};
```

Operações Avançadas e Gerenciamento de Estado

45697056

Gerenciamento de Múltiplos Itens

- **Descrição:** Para otimizar o número de chamadas de rede para o armazenamento local.
  - `multiGet(keysArray)`: Lê vários itens de uma vez. Retorna um array de arrays `[[key1, value1], [key2, value2], ...]`.
  - `multiSet(pairsArray)`: Salva vários pares de uma vez. O formato é `[[key1, value1], [key2, value2], ...]`.
- **Exemplo de Código (MultiGet):**

```
const lerVarios = async () => {
  const chaves = ['@minhaApp:tema', '@minhaApp:usuarioId'];
  try {
    const resultados = await AsyncStorage.multiGet(chaves);
    // resultados pode ser: [['@minhaApp:tema', 'light'], ['@minhaApp:usuarioId', '123']]
    console.log(resultados);
  } catch (error) {
    console.error('Erro MultiGet:', error);
  }
};
```

Integrando com React Hooks e Aplicações Práticas

45697056

Integrando com `useEffect` (Carregamento Inicial)

- **Descrição:** O padrão para garantir que os dados persistidos sejam carregados assim que a tela inicia.
  - Usamos `useEffect` com *array* de dependências vazio (`[]`) para rodar *apenas uma vez* (ao montar o componente).
- **Exemplo de Código (Carregar Tema ao Iniciar):**

```
import React, { useState, useEffect } from 'react';
// ... imports

const TelaConfiguracoes = () => {
  const [tema, setTema] = useState('light'); // Estado inicial

  useEffect(() => {
    const carregarTema = async () => {
      const temaSalvo = await AsyncStorage.getItem('@minhaApp:tema');
      if (temaSalvo) {
        setTema(temaSalvo); // Atualiza o estado com o valor salvo
      }
    };
    carregarTema();
  }, []); // O [] garante que roda só na montagem

  // ... renderiza o componente usando o estado 'tema'
};
```

Integrando com React Hooks e Aplicações Práticas

45697056

Boas Práticas e Segurança

- Pontos cruciais para produção.
  - **Segurança: Nunca** armazene senhas, tokens JWT ou dados PII (Informações Pessoais Identificáveis) sensíveis no `AsyncStorage` (é texto puro no dispositivo). Use `expo-secure-store` para isso.
  - **Performance:** Para grandes volumes de dados ou consultas complexas, use **SQLite/Realm**. O `AsyncStorage` é para *pequenas* quantidades de dados.
  - **Estrutura de Chaves:** Use um prefixo no nome da chave (ex: `'@MeuApp:config'`) para evitar colisões se outras bibliotecas usarem o `AsyncStorage`.

Instalando outros pacotes

45697056

- **@react-navigation/native**: O pacote principal para navegação, permitindo a criação de diferentes telas e a transição entre elas.
- **@react-navigation/stack**: Oferece a funcionalidade de navegação em pilha, onde novas telas são "empilhadas" sobre as anteriores, ideal para fluxos de navegação lineares.
- **react-native-screens**: Otimiza a navegação, utilizando visualizações nativas para cada tela, o que melhora o desempenho.
- **react-native-safe-area-context**: Garante que o conteúdo da tela seja exibido em uma área segura, evitando partes da interface do usuário que podem ser obscurecidas por entalhes ou barras de status em dispositivos modernos.
- **react-native-gesture-handler**: Permite o tratamento de gestos complexos (como deslizar, pinçar, etc.) de forma nativa e performática.

Vamos conhecer o código?

Criando projeto

45697056

- Criando nosso projeto React native + Expo
  - npx create-expo-app --template
  - selecionar template Blank (Typescript)
  - definir nome do projeto
  - entrar na pasta criada (cd "nome da pasta")
  - instalar pacotes
  - npm start
  - escolher plataforma
    - **A** - android
    - **I** - iOS

Explicação comandos

45697056

npx create-expo-app - -template

- Criar um novo projeto Expo usando o template blank ou de sua preferência.
- npx: executa pacotes sem precisar instalá-los globalmente.
- create-expo-app: ferramenta para iniciar projetos Expo.
- –template: lista templates (com React Native + Expo).

Projeto base

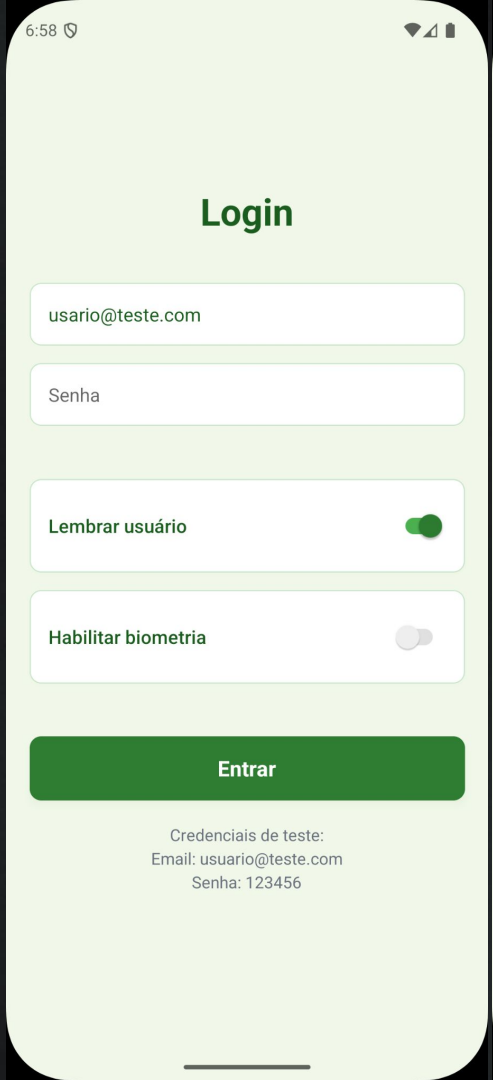
https://github.com/admeneses/async-storage-form

Como executar a primeira vez:

- clonar o projeto
- npm i

Depois de configurado:

npm start + (a plataforma que será executado)



Obrigado

2025-Todos direitos reservados. Reprodução ou divulgação total ou parcial deste documento é expressamente proíbido sem o consentimento formal, por escrito, do Professor Wellington Cidade Silva Tenorio.